

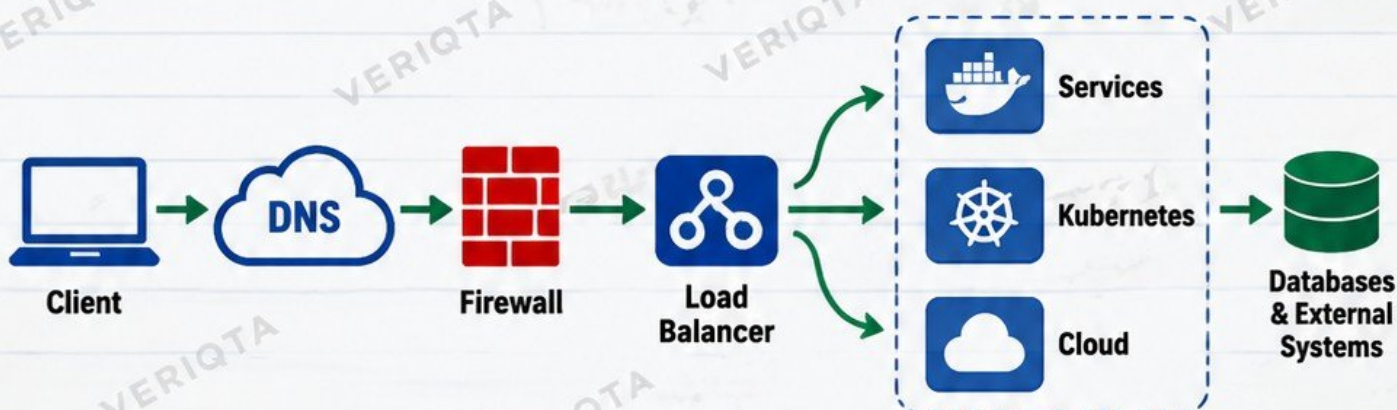
REAL SYSTEMS
REAL INCIDENTS
REAL ENGINEERING



ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

NETWORKING FOR SENIOR DEVOPS ENGINEERS

PRODUCTION NETWORKING | ARCHITECTURE | TROUBLESHOOTING
SECURITY | SCALING | REAL-WORLD SCENARIOS



DESIGN
SCALABLE
NETWORKS



TROUBLESHOOT
COMPLEX
FAILURES



IMPROVE
PERFORMANCE
& RELIABILITY



APPLY
SECURITY
BEST PRACTICES



REAL
PRODUCTION
SCENARIOS

SENIOR ENGINEER NOTEBOOK SERIES

BUILD
OPERATE
SOLVE
AT SCALE

VERIQTA

KNOWLEDGE BUILDS BETTER ENGINEERS

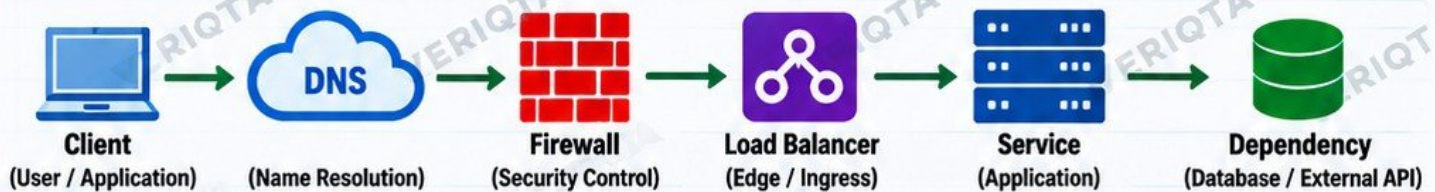
PEOPLE
PLATFORMS
PRODUCTION
PROGRESS

PRODUCTION NETWORK MENTAL MODEL

HOW APPLICATIONS COMMUNICATE IN THE REAL WORLD

UNDERSTAND THE FULL REQUEST PATH, FAILURE POINTS AND ENGINEERING TRADE-OFFS

1. APPLICATION TO APPLICATION REQUEST PATH



2. CONTROL PLANE vs DATA PLANE



Control Plane

- Manages configuration, policies and state
- Examples: DNS management, routing protocols, LB config
- Not in the request path
- Changes here affect how traffic flows



Data Plane

- Handles actual traffic
- Forwards packets from source to destination
- Includes routers, firewalls, load balancers, switches
- Performance, latency and drops happen here

3. NORTH-SOUTH vs EAST-WEST TRAFFIC



North-South Traffic

- Traffic between users (internet) and your network
- Comes in and out of the data center or VPC
- Passes through firewalls, load balancers, ingress
- Usually more tightly controlled
- Examples: user to web app, mobile app to API



East-West Traffic

- Traffic between services inside the network
- Service to service, pod to pod, container to DB
- Often higher volume and lower latency
- Still requires security (NetworkPolicies, firewalls)
- Examples: microservice to database, API to cache

4. WHERE LATENCY AND FAILURES ENTER THE PATH



DNS

- Slow resolution
- DNS server failure
- Incorrect records
- High TTL / caching issues



Network

- Routing issues
- Packet loss
- Congestion
- MTU problems
- Asymmetric routing



Firewall

- Blocked traffic
- Misconfigured rules
- NAT issues
- Connection tracking limits



Load Balancer

- Health check failures
- Backend saturation
- Session issues
- Cross-zone latency
- Connection limits



Service

- Slow response
- Resource saturation
- Connection pool exhaustion
- Application errors



Dependency

- Database latency
- External API failure
- Rate limiting
- Network timeout
- DNS or TLS issues

5. KEY TAKEAWAYS



- A successful request requires every component in the path.
- Failures can happen at any layer, not just the application.
- Control plane changes affect data plane behavior.
- North-south and east-west traffic have different risks.
- Latency is often introduced by DNS, network hops, load balancers or dependencies.
- Always map the full request path when troubleshooting.

6. SENIOR ENGINEER TIP



"When investigating a network issue, look at the entire path, not just the failing component.
Most production outages are caused by interactions between layers, not a single failure."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

TCP IN PRODUCTION

HOW TCP WORKS, WHAT CAN FAIL, AND HOW TO TROUBLESHOOT IT

1. THREE-WAY HANDSHAKE



- Client sends SYN to start a connection.
- Server replies with SYN + ACK.
- Client sends ACK and the connection is established.
- This handshake confirms both sides are reachable and can communicate.

2. TCP CONNECTION LIFECYCLE



- Connection starts with a handshake, moves to ESTABLISHED for data transfer, and then closes with FIN / ACK.
- Both sides close independently.
- Connections can remain open for a long time (keep-alive).

3. IMPORTANT TCP STATES

STATE	MEANING
ESTABLISHED	Connection is active, data is flowing
SYN_SENT	SYN sent, waiting for SYN + ACK
SYN_RECV	SYN received, waiting for ACK
TIME_WAIT	Connection closed, waiting to ensure old packets are not still in the network
CLOSE_WAIT	Remote closed, local application still needs to close
FIN_WAIT	Local side closed, waiting for ACK

4. SYN BACKLOG

- When a server receives SYN packets, they are stored in a SYN backlog queue.
- If the queue is full, new connection attempts may be dropped.
- This can cause connection timeouts, especially during traffic spikes or DDoS attacks.
- Tune with `sysctl` parameters like `net.ipv4.tcp_max_syn_backlog`.

5. RETRANSMISSIONS

- If an ACK is not received, TCP retransmits the data.
- Caused by packet loss, high latency or congestion.
- Too many retransmissions increase latency and can break applications.
- Use `tcpdump` or `nstat` to check retransmission rates.

6. CONGESTION AND FLOW CONTROL

- Congestion control prevents overwhelming the network.
- TCP adjusts the sending rate based on network conditions (e.g. slow start, congestion avoidance, fast recovery).
- Flow control ensures the receiver is not overwhelmed (using a receive window).
- High latency, packet loss or small window sizes reduce throughput.

7. TIME_WAIT AND CLOSE_WAIT

- **TIME_WAIT**: Connection closed, socket remains for a period (usually 60 seconds) to ensure old packets are discarded.
- **CLOSE_WAIT**: Remote side closed, but application has not closed the connection. Can lead to socket leaks.
- Too many **TIME_WAIT** or **CLOSE_WAIT** sockets can exhaust available ports.

8. CONNECTION EXHAUSTION

- Each outbound connection uses an ephemeral port.
- If many connections are opened (and not closed), the server can run out of ports.
- Symptoms: connection failures, timeouts, high **TIME_WAIT** count.
- Fix: close connections properly, use connection pooling, increase ephemeral port range if needed.

9. USEFUL COMMANDS

# Show current connections	
<code>ss -s</code>	# Summary of socket usage
<code>ss -tanp</code>	# Show TCP connections with process
# Capture TCP traffic	
<code>tcpdump -i eth0 tcp -nn</code>	# Capture TCP packets
# TCP statistics	
<code>nstat -az grep Tcp</code>	# Show TCP counters (retransmits, drops, etc.)

SENIOR ENGINEER TIP

"High retransmissions, growing **TIME_WAIT** count or many connections in **CLOSE_WAIT** are early signals of network or application issues. Monitor them before they become an incident."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

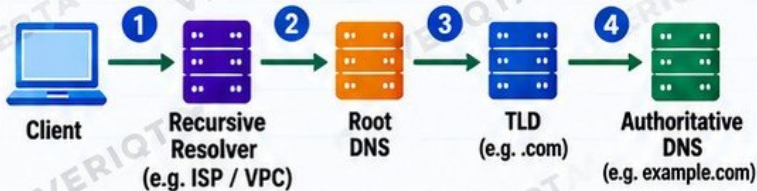
Build
Operate
Improve
Repeat

DNS FAILURE ENGINEERING

HOW DNS WORKS, WHY IT FAILS, AND HOW TO DEBUG IT

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. RECURSIVE RESOLUTION PATH



- Client asks recursive resolver for `www.example.com`
- Resolver checks cache, if not found it queries Root DNS
- Root returns TLD server (e.g. `.com`)
- Resolver queries TLD, gets authoritative DNS
- Authoritative DNS returns the final IP address
- Resolver caches the result and returns it to the client

2. AUTHORITATIVE DNS

- Holds the actual DNS records for your domain.
- Responsible for answering queries about your domain.
- Managed by your DNS provider (e.g. Route 53, Cloudflare, GoDaddy).
- Returns records like A, AAAA, CNAME, MX, TXT etc.
- If authoritative DNS is down or misconfigured, your domain will not resolve.

3. TTL AND CACHING

- TTL (Time To Live) tells resolvers how long to cache a record.
- Higher TTL = less DNS traffic but slower to pick up changes.
- Lower TTL = faster changes but more DNS queries.
- Records are cached at multiple layers: client OS, local resolver, ISP, and sometimes.

Example:
TTL = 300
(5 minutes)

4. NEGATIVE CACHING

- When a domain does not exist (NXDOMAIN), resolvers cache the negative response for a period (based on SOA TTL).
- This means the domain may continue to appear as not found even after you create it.
- Negative caching can cause confusion during DNS changes or new deployments.

5. RESOLVER BEHAVIOR

- Checks local cache first.
- If not found, performs recursive queries.
- Respects TTL and negative caching.
- May use multiple upstream DNS servers.
- Can be affected by network latency, firewall rules or DNS filtering.

6. SPLIT-HORIZON DNS

- Different DNS responses based on the source (internal vs external).
- Used to return private IPs inside your network and public IPs outside.
- Common in hybrid/cloud environments.
- Misconfiguration can cause services to be unreachable internally or externally.

7. DNS LATENCY AND TIMEOUTS

- High DNS latency increases application response time.
- Caused by slow resolvers, network issues, or overloaded authoritative servers.
- Timeouts can result in application failures even if the service is healthy.
- Tune resolver timeouts and retries for critical applications.

8. NXDOMAIN AND SERVFAIL

RESPONSE	MEANING
NXDOMAIN	Domain does not exist. Often cached (negative caching).
SERVFAIL	DNS server failed to process the request. Usually a server issue.
NOERROR	Successful response.
TIMEOUT	No response received.

10. REAL PRODUCTION INCIDENT

Scenario: Application appears down.

Symptoms:

- Users cannot access the application.
- Application logs show connection failures.
- Ping by IP works, but domain name fails.

Investigation:

- Check DNS resolution with `dig`.
- Look for NXDOMAIN or SERVFAIL.
- Verify DNS from different networks.
- Check recent DNS changes.
- Inspect resolver logs and network traffic.

Root Cause (example):

- Authoritative DNS misconfiguration caused SERVFAIL responses.

Fix:

- Correct DNS configuration.
- Reduce negative TTL for faster recovery.
- Monitor DNS health and set up alerts.

9. DEBUGGING DNS ISSUES

DIG (Query DNS)

```
dig www.example.com
dig @8.8.8.8 example.com
dig +trace example.com
```

- Check DNS response
- Query specific server
- Trace full resolution path

RESOLVCTL (Systemd Resolver)

```
resolvectl status
resolvectl query example.com
```

- Check resolver configuration
- Test DNS resolution
- View current DNS servers

TCPDUMP (Capture DNS Traffic)

```
sudo tcpdump -i any port 53
sudo tcpdump -i eth0 port 53 -nn
```

- Capture DNS queries and responses
- Useful to see if queries leave the host
- Check for blocked or dropped traffic



SENIOR ENGINEER TIP

"When an application is unreachable, always check DNS early. A healthy service with bad DNS looks the same as a real outage."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

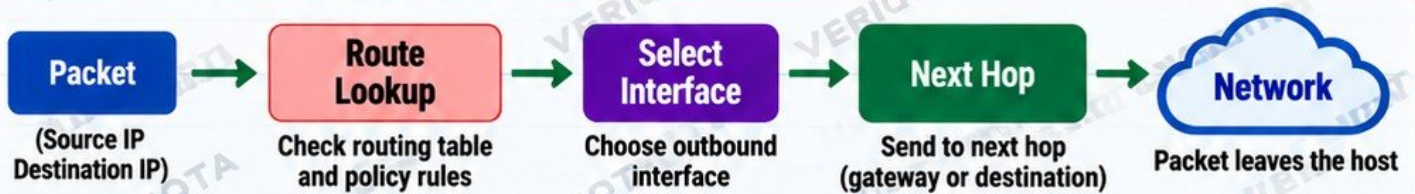


@veriqta

ROUTING AND THE LINUX NETWORK PATH

HOW LINUX DECIDES WHERE TO SEND PACKETS

1. LINUX NETWORK PATH (SIMPLIFIED FLOW)



2. ROUTING TABLES

- Define where packets go based on destination IP.
- Each route has a destination, next hop and interface.
- The main table is usually table 254 (main).
- View routes: **ip route**

```
# Show routing table
ip route
# Show a specific network
ip route get 8.8.8.8
```

3. LONGEST-PREFIX MATCHING

- Linux uses longest-prefix match to choose the most specific route.
- A /24 route is preferred over a /16 or /8.
- Default route (0.0.0.0/0) is used only if no more specific route exists.

Destination	Next Hop	Prefix
10.0.1.0	192.168.1.1	/24
10.0.0.0	192.168.1.1	/16
0.0.0.0	192.168.1.1	/0

4. DEFAULT ROUTES

- Used when no more specific route is found.
- Typically points to a gateway.
- Only one default route is normally used, but multiple can exist with metrics.

```
# Show default route
ip route | grep default
# Add a default route
ip route add default via 192.168.1.1 dev eth0
```

5. POLICY ROUTING

- Allows routing based on source IP, user, mark or other criteria.
- Useful in multi-homed servers, VPNs and complex architectures.
- Uses routing rules and multiple routing tables.

```
# Show policy rules
ip rule show
# Add a rule for a specific source IP
ip rule add from 10.0.1.10 table 100
```

6. MULTIPLE INTERFACES

- Servers often have multiple interfaces (e.g. public, private, VPN).
- Each interface can have its own routes.
- Verify interface IPs and status.

```
# Show interfaces
ip addr show
# Show routes per interface
ip route show dev eth0
# Check link status
ip link show
```

7. ASYMMETRIC ROUTING

- Happens when request and response take different paths.
- Can cause connection issues, dropped packets or firewall problems.
- Common in multi-homed or peered networks.
- Use consistent routing or policy routing to fix.

8. REVERSE-PATH FILTERING (RPF)

- Validates that the source IP is reachable via the incoming interface.
- Helps prevent IP spoofing.
- Can drop legitimate traffic in asymmetric routing setups.
- Check and configure:

```
# Check RPF setting (0=off, 1=strict, 2=loose)
sysctl net.ipv4.conf.all.rp_filter
# Set to loose mode (recommended for multi-homed)
sysctl -w net.ipv4.conf.all.rp_filter=2
```

9. USEFUL COMMANDS

COMMAND	PURPOSE
ip route	Show routing table
ip rule	Show policy routing rules
ip addr	Show interface IP addresses
ip neigh	Show ARP/neighbor table
ip route get <IP>	See which route will be used
ss -r	Show routing cache info
ip link	Show interface status

10. KEY TAKEAWAYS

- Linux uses longest-prefix match for routing decisions.
- Default routes handle unknown destinations.
- Policy routing is essential for complex setups.
- Asymmetric routing can break connections.
- Reverse-path filtering helps security but can drop valid traffic.
- Always verify routes, rules and interface configuration when troubleshooting.

SENIOR ENGINEER TIP

"When a network issue occurs, always check the actual route the packet is taking with **ip route get <destination>** instead of guessing."

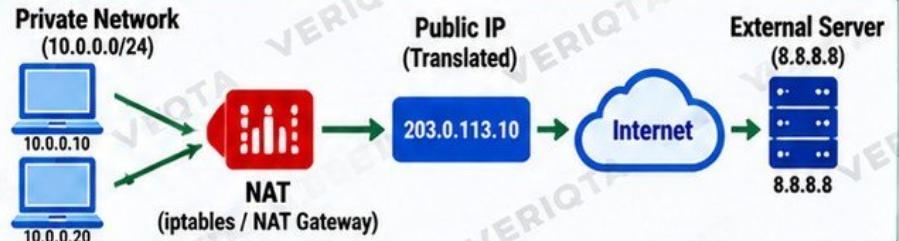
NAT, SNAT, DNAT AND PORT EXHAUSTION

HOW ADDRESS TRANSLATION WORKS, WHAT CAN FAIL, AND HOW TO FIX IT

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. NAT OVERVIEW AND TRANSLATION MECHANICS

- NAT (Network Address Translation) modifies IP addresses and/or ports in packet headers.
- It allows private networks to access external networks and expose services securely.
- Translation is stateful and tracked in a connection table (conntrack).



2. SOURCE NAT (SNAT)

- Changes the source IP (and often port) of outbound traffic.
- Used for private instances to access the internet.
- Common in NAT gateways and egress routers.

Example (iptables SNAT)

```
iptables -t nat -A POSTROUTING \
-s 10.0.0.0/24 -o eth0 -j SNAT \
--to-source 203.0.113.10
```

3. DESTINATION NAT (DNAT)

- Changes the destination IP (and often port) of incoming traffic.
- Used to expose internal services (e.g. load balancers, port forwarding).

Example (iptables DNAT)

```
iptables -t nat -A PREROUTING \
-p tcp --dport 80 -j DNAT \
--to-destination 10.0.0.10:8080
```

4. CONNECTION TRACKING

- NAT is stateful. A conntrack table keeps track of active connections.
- Each entry maps: source, destination, ports, protocol and state.
- If the table is full, new connections fail.
- You can inspect conntrack entries and limits.

View conntrack entries

```
conntrack -L | head
conntrack -S
```

5. EPHEMERAL PORTS

- Outbound connections use ephemeral (source) ports, typically 32768–60999 (on most Linux systems).
- Each unique destination requires a unique source port.
- High outbound traffic can exhaust available ports.

Check port range

```
cat /proc/sys/net/ipv4/ip_local_port_range
```

6. NAT GATEWAY BEHAVIOR

- Uses a public IP (or pool of IPs) to translate many private IPs.
- Each outbound connection consumes one ephemeral port on the NAT IP.
- Managed NAT gateways have limits on connections and bandwidth.
- Cross-AZ or cross-region traffic can increase cost and latency.

7. PORT EXHAUSTION

- Happens when all ephemeral ports are in use for a destination (IP:port).
- Symptoms: new connections fail, timeouts, intermittent errors.
- Common with high concurrency, short-lived connections or many pods/containers.

Check active connections to a destination

```
ss -s
ss -tan | grep :443 | wc -l
```

8. CONNTRACK SATURATION

- When conntrack table is full, new connections are dropped.
- Can cause intermittent failures under load.
- Monitor usage and increase limits if needed.
- Tune timeouts for short-lived connections.

Check conntrack usage

```
conntrack -S
cat /proc/sys/net/netfilter/nf_conntrack_max
```

9. SCALING IMPLICATIONS

- Use multiple NAT IPs or NAT gateways for high-scale workloads.
- Distribute traffic across multiple destinations when possible.
- Reuse connections (keep-alive, connection pooling).
- Tune application timeouts and retry logic.
- Consider private connectivity (VPC peering, Direct Connect) to reduce NAT load.

10. REAL PRODUCTION INCIDENT

Scenario: Outbound traffic fails only under peak load.

Symptoms:

- Applications time out when calling external APIs.
- Works during low traffic, fails at peak.
- No application errors, just connection timeouts.

Root Cause: NAT port exhaustion on the NAT gateway IP.

Fix:

- Increase NAT IPs or use multiple NAT gateways.
- Reuse connections and tune timeouts.
- Monitor port and conntrack usage.



SENIOR ENGINEER TIP

"NAT is a critical dependency. Monitor port usage, conntrack, and error rates before you reach the limits. Design for scale, not just for today's traffic."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

Build
Operate
Improve
Repeat

LOAD BALANCING UNDER FAILURE

HOW LOAD BALANCERS WORK, WHAT CAN FAIL, AND HOW TO DESIGN FOR RESILIENCE

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. LAYER 4 vs LAYER 7

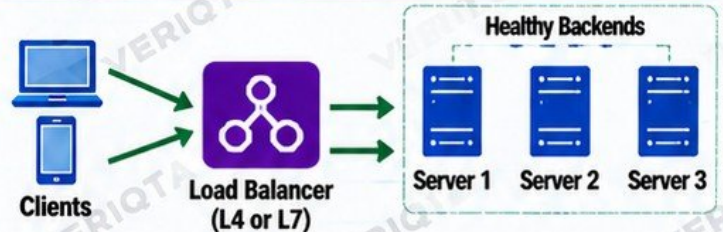
LAYER 4 (TRANSPORT)

- Works at TCP/UDP level.
- Uses IP and port only.
- Faster and simpler.
- Cannot see application data.
- Common in high-performance and low-latency setups.
- Examples: AWS NLB, TCP LB, HAProxy (L4), IPVS.

LAYER 7 (APPLICATION)

- Works at HTTP/HTTPS level.
- Can inspect headers, paths, cookies, hostnames.
- Supports advanced routing rules.
- Slower than L4 but more flexible.
- Examples: ALB, NGINX, Envoy, HAProxy (L7), Traefik.

2. HOW A LOAD BALANCER WORKS



- 1 Client sends request to load balancer (DNS or IP).
- 2 Load balancer checks health and selects a backend.
- 3 Request is forwarded to the chosen server.
- 4 Response is returned to the client (through the load balancer).

3. HEALTH CHECKS

- Used to detect healthy and unhealthy backends.
- Can be TCP, HTTP, HTTPS or custom checks.
- If a backend fails health checks, it is removed from rotation.
- Misconfigured health checks can remove healthy servers or keep unhealthy ones.

Example (HTTP health check - NGINX)

```
location /health {
    return 200 "OK";
}
```

4. CONNECTION DISTRIBUTION

- Algorithms decide which backend gets the next request.
- Common algorithms:
 - Round Robin
 - Least Connections
 - Weighted Round Robin
 - IP Hash (source IP)
 - Least Response Time
- Choose an algorithm based on your application and workload characteristics.

5. STICKY SESSIONS

- Keeps a client connected to the same backend (using cookies or source IP).
- Useful for stateful applications (e.g. shopping carts, sessions).
- Can cause uneven load distribution.
- Consider using external session storage instead of sticky sessions.

6. CONNECTION DRAINING

- Allows existing connections to complete before removing a server from rotation.
- Used during deployments or scale-in events.
- Prevents dropped connections.
- Configured with a drain timeout (e.g. 30-300 seconds).

7. SLOW-START BEHAVIOR

- Gradually increases the number of requests sent to a new backend.
- Prevents new servers from being overwhelmed.
- Useful after deployments or when bringing a server back online.
- Supported by many load balancers (e.g. ALB, NGINX, HAProxy).

8. CROSS-ZONE TRAFFIC

- Load balancer can send traffic to backends in different availability zones (AZs).
- Improves availability but can increase latency and cost.
- Some setups allow zone-aware routing to keep traffic local.
- Consider trade-offs between resilience, latency and data transfer costs.

9. BACKEND SATURATION

- Happens when servers cannot handle more requests (CPU, memory, connections).
- Leads to high latency, timeouts and errors.
- The load balancer may still see the server as healthy if health checks are too simple.
- Monitor backend metrics (CPU, memory, request rate, response time) and set realistic limits.

10. FAILURE AMPLIFICATION

- A small failure can be amplified by the load balancer (e.g. sending retries to an already overloaded backend).
- Can lead to cascading failures.
- Use proper health checks, retries with backoff, and circuit breakers.
- Monitor for retry storms and high error rates.

11. KEY TRADE-OFF

BALANCING EFFICIENCY	VS	FAILURE ISOLATION
• Lower latency • Better resource utilization • Higher throughput		• Limits blast radius • May use fewer backends • Can increase cost and latency • More resilient to failures

SENIOR ENGINEER TIP

"A load balancer hides failures, but it can also amplify them. Design your health checks, retry logic and scaling strategy to assume something will fail."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

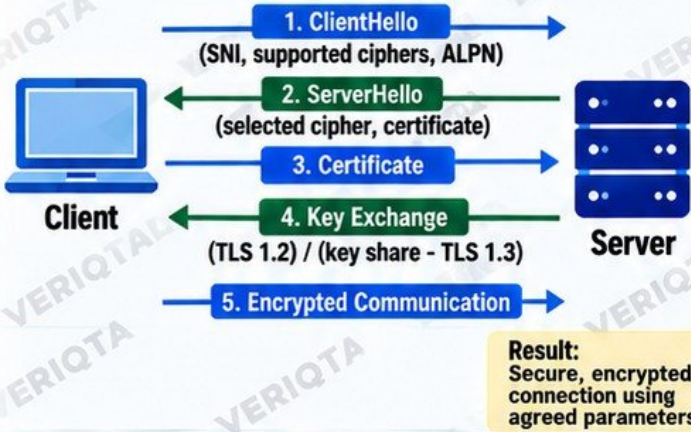
KNOWLEDGE BUILDS BETTER ENGINEERS

Build
Operate
Improve
Repeat

TLS AND CERTIFICATE FAILURE ANALYSIS

HOW TLS WORKS, WHAT CAN FAIL, AND HOW TO TROUBLESHOOT IT

1. TLS HANDSHAKE (SIMPLIFIED FLOW)



2. CERTIFICATE CHAIN

- Server sends its certificate (leaf) and intermediate certificates.
- Client builds the chain up to a trusted root CA.
- Chain must be complete and in the correct order.
- If any certificate is missing, expired, or untrusted, verification fails.



Check certificate chain:

```
openssl s_client -connect example.com:443 -showcerts
```

View certificate details:

```
openssl x509 -in cert.pem -text -noout
```

3. SNI (SERVER NAME INDICATION)

- Client sends the domain name (SNI) in the ClientHello.
- Allows the server to present the correct certificate (multiple domains on one IP).
- If SNI is missing or incorrect, the server may return the wrong certificate.

Test with and without SNI:

With SNI (default)

```
openssl s_client -connect example.com:443 -servername example.com
```

Without SNI

```
openssl s_client -connect example.com:443
```

4. ALPN (APPLICATION LAYER PROTOCOL NEGOTIATION)

- Client and server negotiate the application protocol (e.g. HTTP/1.1 or HTTP/2).
- Improves performance (HTTP/2, gRPC).
- If there is no common protocol, the handshake may fail.

Check negotiated protocol:

```
openssl s_client -connect example.com:443 -alpn h2 -servername example.com | grep ALPN
```

Or with curl:

```
curl -v https://example.com
```

5. CERTIFICATE VERIFICATION

- Client verifies:
 - Certificate is valid and not expired.
 - Domain name matches (CN or SAN).
 - Chain is trusted by a trusted CA.
 - Not revoked (CRL/OCSP, if used).
- Any mismatch will cause the handshake to fail.

Common verification errors:

- certificate has expired
- certificate is not yet valid
- unable to get local issuer certificate
- hostname mismatch

6. mTLS (MUTUAL TLS)

- Both client and server present certificates.
- Used for high-security environments (service to service).
- Client certificate is verified by the server.
- Common in Kubernetes, service meshes and internal APIs.

Test mTLS connection:

```
openssl s_client -connect example.com:443 -cert client.pem -key client.key -CAfile ca.pem
```

7. EXPIRATION

- Certificates have a validity period (start and end date).
- Expired certificates will be rejected by clients.
- Monitor certificate expiration to prevent outages.

Check expiration date:

```
openssl x509 -in cert.pem -noout -dates
```

8. CIPHER / PROTOCOL MISMATCH

- Client and server must support a common TLS version (e.g. TLS 1.2, TLS 1.3).
- If there is no common cipher/protocol, the handshake fails.
- Older clients may not support modern ciphers.

Test supported ciphers and protocols:

```
openssl s_client -connect example.com:443 -tls1_2 # force TLS 1.2
openssl s_client -connect example.com:443 -tls1_3 # force TLS 1.3
```

9. TLS TERMINATION

- TLS can be terminated at a load balancer, reverse proxy, or ingress (e.g. NGINX, HAProxy, ALB).
- Traffic between the proxy and backend can be HTTP or re-encrypted with TLS.
- Misconfiguration can lead to certificate errors, wrong certificates, or insecure backend traffic.

10. USEFUL COMMANDS

COMMAND	PURPOSE
<code>openssl s_client</code>	Test TLS connection and view certificate details
<code>curl -v</code>	Test HTTPS and see verbose output
<code>openssl x509</code>	View certificate information
<code>openssl verify</code>	Verify certificate chain
<code>nmap --script ssl-cert</code>	Check certificate details (nmap)

11. REAL PRODUCTION INCIDENT

Scenario: TCP works, HTTPS fails.

Symptoms:

- You can ping and connect to port 443 (TCP).
- HTTPS requests fail with SSL/TLS errors.
- Application appears down, but only HTTPS is affected.

Possible causes:

- Expired or invalid certificate.
- Hostname mismatch (SNI issue).
- Incomplete certificate chain.
- Unsupported TLS version or cipher.
- TLS misconfiguration at load balancer or proxy.



SENIOR ENGINEER TIP

"When TCP works but HTTPS fails, always check the certificate, SNI, and TLS handshake before assuming the service is down."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

PROXIES, REVERSE PROXIES AND NETWORK INTERMEDIARIES

HOW THEY WORK, WHY THEY MATTER, AND HOW TO TROUBLESHOOT THEM

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. FORWARD vs REVERSE PROXIES

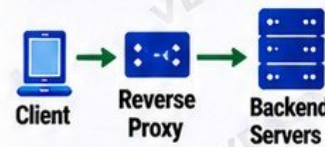
FORWARD PROXY

- Sits in front of clients.
- Used by internal users to access external resources.
- Hides the client's IP address.
- Common in corporate networks for security, filtering and monitoring.



REVERSE PROXY

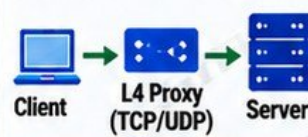
- Sits in front of servers.
- Receives client requests and forwards them to backend servers.
- Hides the backend servers.
- Used for load balancing, SSL termination, security and caching.



2. L4 vs L7 PROXIES

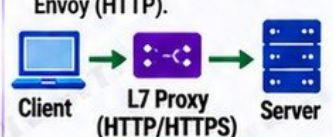
LAYER 4 PROXY (TRANSPORT)

- Works at TCP/UDP level.
- Routes based on IP and port.
- Does not inspect application data.
- Faster and lower overhead.
- Examples: HAProxy (TCP mode), AWS NLB, Envoy (TCP).



LAYER 7 PROXY (APPLICATION)

- Works at HTTP/HTTPS level.
- Can inspect and modify requests and responses.
- Routes based on headers, paths, cookies, hostnames.
- Supports advanced features (caching, auth, rewriting).
- Examples: NGINX, HAProxy (HTTP), Envoy (HTTP).



3. ENVOY, HAPOXY AND NGINX CONCEPTS

ENVOY

- High-performance L7 proxy.
- Common in service meshes (e.g. Istio).
- Supports advanced routing, observability and retries.
- Used in modern cloud-native environments.

HAPOXY

- High-performance L4 and L7 proxy.
- Widely used for load balancing.
- Known for stability and rich feature set.
- Supports health checks, retries and SSL termination.

NGINX

- Popular L7 proxy and web server.
- Used for reverse proxy, load balancing, SSL termination.
- Simple and flexible configuration.
- Common in traditional and modern environments.

4. HEADERS AND CLIENT IP PRESERVATION

- Proxies can hide the real client IP.
- Use headers to preserve client information:
 - X-Forwarded-For (adds client IP)
 - X-Real-IP
 - Forwarded (standard header)
- Configure your application to trust these headers.

Example (NGINX):

```

proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header Forwarded $proxy_add_forwarded_for;
  
```

5. CONNECTION REUSE

- Proxies reuse connections to improve performance.
- Keeps backend connections open for multiple requests.
- Reduces latency and resource usage.
- Can hide backend failures because old connections remain healthy.
- Be aware of connection pool limits.

6. PROXY BUFFERING

- Proxies can buffer requests and responses.
- Helps with performance and handles slow backends.
- Can cause unexpected delays.
- Large buffers can hide real-time issues.
- Tune buffer sizes based on workload.

7. TIMEOUT CHAINS

- Each layer (client, proxy, load balancer, application, backend) has its own timeout.
- If timeouts are not aligned, you can get:
 - Premature retries
 - Unexpected connection drops
 - 502/504 errors
- Always configure consistent timeouts across all layers.

8. RETRY BEHAVIOR

- Proxies can automatically retry failed requests.
- Helps with transient failures.
- Can amplify load if the backend is down.
- Use sensible retry limits and backoff strategy.
- Consider idempotency when using retries (avoid retrying non-idempotent requests).

9. HOW INTERMEDIARIES HIDE THE REAL FAILURE DOMAIN

- Proxies and intermediaries can mask where the real failure is happening.
- A 502/504 error may come from the proxy, not the application.
- Retries and buffering can make a failing service look healthy.
- Multiple layers (CDN → LB → Proxy → App → DB) can make troubleshooting difficult.
- Always check logs and metrics at each layer.
- Use correlation IDs and tracing to find the real source of the failure.

Common example:

Users see 502 Bad Gateway, but the real issue is a slow database causing the application to timeout.



SENIOR ENGINEER TIP

"When troubleshooting, always check every intermediary layer. The error you see is often not where the problem actually started."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

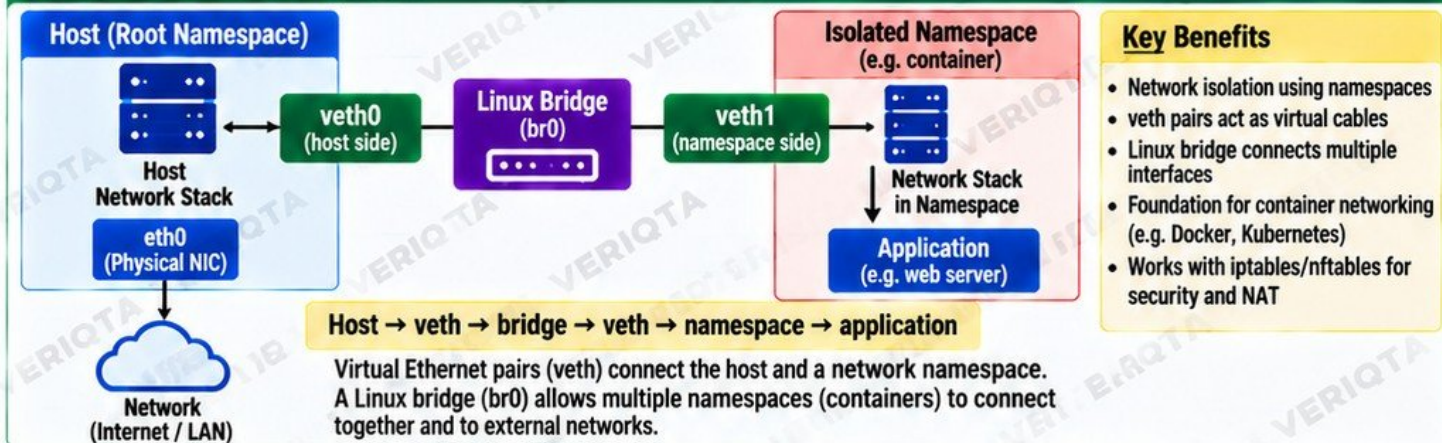
Build
Operate
Improve
Repeat

LINUX NETWORK NAMESPACES, VETH AND BRIDGES

HOW LINUX ISOLATES, CONNECTS AND ROUTES NETWORKS

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. OVERVIEW ARCHITECTURE



2. NETWORK NAMESPACES

- Provide isolated network stacks (routing tables, interfaces, iptables, etc).
- Each namespace has its own interfaces, routes and firewall rules.
- Commonly used for containers and network testing.

Useful commands:

```
# List namespaces
ip netns list

# Create a namespace
ip netns add ns1

# Run a command in a namespace
ip netns exec ns1 ip addr
```

3. VIRTUAL ETHERNET PAIRS (veth)

- A veth pair is like a virtual network cable.
- Packets sent on one end appear on the other end.
- One end is usually in the host namespace and the other in a target namespace.
- Used to connect namespaces to bridges or directly to each other.

Useful commands:

```
# Create a veth pair
ip link add veth0 type veth peer name veth1

# Move one end to a namespace
ip link set veth1 netns ns1

# Bring interfaces up
ip link set veth0 up
ip netns exec ns1 ip link set veth1 up
```

4. LINUX BRIDGES

- A Linux bridge works like a Layer 2 switch.
- Connects multiple interfaces (physical, veth, tap).
- Forwards traffic between connected interfaces based on MAC addresses.
- Used in container networking (e.g. Docker bridge, Kubernetes CNI).

Useful commands:

```
# Create a bridge
ip link add br0 type bridge

# Add interface to bridge
ip link set veth0 master br0

# Bring bridge up
ip link set br0 up

# Show bridge information
bridge link
```

5. NAMESPACE ROUTING

- Each namespace has its own routing table.
- You must configure routes inside the namespace if it needs external access.
- Namespaces can have default routes via the host or a bridge.

Example:

```
# Set IP address inside namespace
ip netns exec ns1 ip addr add 10.0.0.2/24 dev veth1

# Add default route
ip netns exec ns1 ip route add default via 10.0.0.1

# Show routes
ip netns exec ns1 ip route
```

6. IPTABLES / NFTABLES INTERACTION

- Namespaces have their own iptables/nftables rules.
- Host firewall rules can also affect traffic (e.g. NAT, forwarding).
- Used for network isolation, NAT and security policies.
- Container runtimes configure these rules automatically.

Check rules:

```
# In host namespace
iptables -L -n -v

# In a namespace
ip netns exec ns1 iptables -L -n -v

# nftables example
nft list ruleset
```

7. CONTAINER NETWORKING FOUNDATIONS

- Containers use network namespaces for isolation.
- veth pairs connect containers to a bridge.
- Bridges connect containers to each other and to the host network.
- Docker, Kubernetes and other runtimes build on these Linux primitives.
- Understanding these components helps with troubleshooting and custom setups.

Common container network modes:

- Bridge mode (default)
- Host mode
- None (isolated)
- Custom bridge networks (user-defined)

8. KEY TAKEAWAYS

- Network namespaces isolate network stacks.
- veth pairs connect namespaces.
- Linux bridges enable multiple interfaces to communicate.
- Routing, firewall and NAT must be configured correctly.
- These are the building blocks for container networking.



SENIOR ENGINEER TIP

"When troubleshooting container networking, always check the namespace, veth pair, bridge, routes and firewall rules in that order."

9. USEFUL COMMAND REFERENCE

COMMAND	PURPOSE
ip netns	Manage network namespaces
ip link	Manage network interfaces (veth, bridge)
bridge	Show and manage Linux bridges
ip addr	Show IP addresses
ip route	Show and add routes
iptables / nft	Manage firewall/NAT rules



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



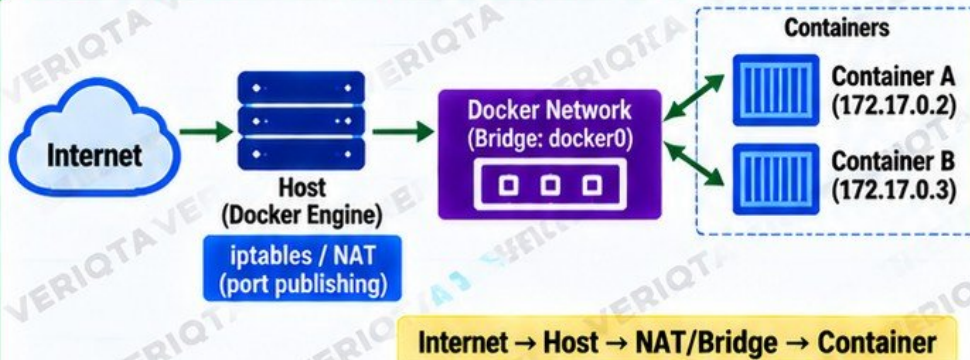
@veriqta

DOCKER NETWORKING INTERNALS

HOW DOCKER CONNECTS CONTAINERS, EXPOSES SERVICES, AND MOVES PACKETS

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. OVERVIEW: DOCKER NETWORKING ARCHITECTURE



Key Points

- Docker uses Linux networking features (namespaces, veth, bridge, iptables).
- Each container gets its own network namespace and IP address.
- Containers connect through a bridge by default.
- NAT is used for outbound traffic and port publishing.
- Docker provides internal DNS for container name resolution.
- Other drivers (host, overlay, macvlan) are used for different use cases.

2. BRIDGE NETWORKING (DEFAULT)

- Uses a Linux bridge (docker0).
- Containers connect via veth pairs.
- Provides internal network for containers on the same host.
- Isolated from the external network by default.
- Common for single-host Docker setups.

Useful commands:

```
docker network ls
docker network inspect bridge
ip link show docker0
brctl show
```

3. CONTAINER NAMESPACES

- Each container has its own network namespace (isolated network stack).
- Namespace contains its own interfaces, routing table, and firewall rules.
- Connected to the host via a veth pair (one end in host, one end in container).
- Provides isolation and security.

Useful commands:

```
docker inspect <container_id>
ls -l /var/run/docker/netns
ip netns list
nsenter -t <pid> -n ip addr
```

4. CONTAINER IP ASSIGNMENT

- Containers get an IP from Docker's built-in IPAM (default bridge subnet 172.17.0.0/16).
- IP is assigned when the container starts.
- IP can change when the container is recreated.
- Use static IPs or custom networks when needed.

Useful commands:

```
docker network inspect bridge
docker inspect <container_id> | grep IP
ip addr show
```

5. NAT AND PORT PUBLISHING

- Docker uses iptables NAT (MASQUERADE) for outbound traffic.
- Port publishing maps host ports to container ports (DNAT).
- Allows external access to containerized services.
- Traffic flows: Host:Port → Container:Port.

Useful commands:

```
docker run -p 8080:80 nginx
iptables -t nat -L -n | grep docker
docker port <container_id>
```

6. DOCKER DNS

- Docker provides an internal DNS server (127.0.0.11) for container name resolution.
- Containers can reach each other using container names (not just IPs).
- Works within the same Docker network.
- Simplifies service discovery for multi-container apps.

Example:

```
docker run --name web nginx
docker run --name app --link web nginx
# In container:
ping web
curl http://web
```

7. HOST NETWORKING

- Container shares the host's network namespace.
- No network isolation (uses host IP directly).
- Higher performance (no NAT).
- Common for specific use cases (e.g. monitoring, high performance).

Example:

```
docker run --network host nginx
```

- No port mapping needed.
- Be careful: reduces isolation.

8. OVERLAY NETWORKING

- Used for multi-host Docker environments (e.g. Docker Swarm).
- Creates a virtual network across multiple hosts.
- Allows containers on different hosts to communicate.
- Managed by Docker and uses VXLAN under the hood.
- Common in container orchestration platforms.

9. CROSS-CONTAINER TRAFFIC

- Containers on the same network can communicate directly (using container names).
- Traffic between networks requires routing or multiple network attachments.
- Use custom networks for better control and isolation.
- Apply firewall rules (iptables/nftables) when needed.

10. PACKET-PATH TROUBLESHOOTING

- 1 Check container IP and network: `docker inspect`
- 2 Verify connectivity: `ping`, `curl`, `nc`
- 3 Check Docker networks: `docker network inspect`
- 4 Inspect iptables rules: `iptables -t nat -L`
- 5 Trace packets: `tcpdump` on host or container
- 6 Verify DNS: `dig @127.0.0.11 <container_name>`
- 7 Check routing: `ip route`, `ip netns exec`



SENIOR ENGINEER TIP

"When troubleshooting container networking, always follow the packet path step by step and verify each layer: namespace → interface → routing → NAT → firewall → destination."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

Build
Operate
Improve
Repeat

KUBERNETES POD NETWORKING

HOW PODS CONNECT, HOW TRAFFIC FLOWS, AND HOW TO TROUBLESHOOT IT

1. POD NETWORK MODEL

- Each pod gets its own IP address.
- Pods can communicate with each other directly (no NAT).
- All containers in a pod share the same network namespace (same IP, same network interfaces).
- The pod network is a flat network across the cluster.
- Handled by a Container Network Interface (CNI) plugin.
- Kubernetes does not use Docker networking for pod networking (it uses a CNI).



Pod A → Node A → CNI Network → Node B → Pod B

Pods communicate across nodes using the CNI network (overlay or routed).

Key Takeaways

- Every pod has a unique IP.
- Pods can talk directly across the cluster.
- CNI plugin sets up the network (not Docker).
- The network must work across all nodes.

2. CNI RESPONSIBILITIES

- Assign IP addresses to pods.
- Create network interfaces in the pod network namespace.
- Set up routes and rules for pod-to-pod and pod-to-external traffic.
- Handle network policies (if supported).
- Manage encapsulation (e.g. VXLAN, Geneve) for overlay networks.
- Integrate with Kubernetes (through CNI spec).

Popular CNI plugins:

Calico, Flannel, Cilium, Weave, Canal

3. POD-TO-POD TRAFFIC

- Pods can communicate directly using their IP addresses.
- If on the same node, traffic stays on the node (no encapsulation).
- If on different nodes, traffic goes through the CNI network (overlay or routed).
- Network policies can allow or deny pod-to-pod traffic.

Example (from inside a pod):

```
ping 10.244.2.20
curl http://10.244.2.20:8080
```

4. NODE-TO-NODE TRAFFIC

- When pods are on different nodes, traffic must be routed across nodes.
- CNI uses one of these methods:
 - Overlay networking (e.g. VXLAN, Geneve, IP-in-IP)
 - Routed networking (BGP, native routing)
- Node-to-node communication happens over the underlay network (e.g. cloud VPC, data center network).
- Firewalls and security groups must allow CNI traffic (e.g. UDP 4789 for VXLAN).

Example (check node routes):

```
ip route | grep 10.244
```

5. ROUTES AND OVERLAYS

- Overlay networks encapsulate pod traffic inside another packet (e.g. VXLAN).
- Routes tell the node where to send pod traffic.
- Each node knows how to reach pod CIDRs on other nodes.
- Some CNIs (e.g. Calico) can use BGP instead of overlays.

Example (view routes):

```
ip route show
ip route | grep 10.244
```

6. ENCAPSULATION

- Overlay networks add an extra header (e.g. VXLAN, Geneve, IP-in-IP).
- This allows pod traffic to travel across the underlay network.
- Encapsulation increases packet size.
- Used when direct routing is not available.

Packet Encapsulation (VXLAN Example)



7. MTU IMPLICATIONS

- Encapsulation adds extra bytes to the packet.
- This reduces the effective MTU for pod traffic.
- If MTU is too high, packets may be fragmented or dropped.
- Can cause issues like slow connections, timeouts, or failed transfers.
- Adjust MTU on nodes and CNI as needed (e.g. 1450 instead of 1500).

Check MTU:

```
ip link show
ip link show <interface>
```

8. CNI FAILURE MODES

- Pods cannot get an IP address (CNI not running).
- Pod-to-pod communication fails.
- Nodes cannot reach each other (routing/firewall issue).
- Incorrect MTU (causes dropped packets).
- CNI agent crashes or misconfiguration.
- Network policies blocking traffic.
- DNS issues inside the cluster (if provided by CNI or CoreDNS).

9. USEFUL COMMANDS

COMMAND	PURPOSE
<code>ip a</code>	Show network interfaces
<code>ip route</code>	Show routing table
<code>ip link</code>	Show link/MTU details
<code>kubectl get pods -o wide</code>	Show pod IPs and nodes
<code>kubectl get nodes -o wide</code>	Show node IPs
<code>kubectl get svc</code>	Show services
<code>tcpdump -i <interface></code>	Capture network traffic

10. KEY TAKEAWAYS

- Pods have their own IP addresses.
- CNI handles pod networking.
- Pods can communicate directly.
- Traffic may be routed or encapsulated across nodes.
- Encapsulation reduces effective MTU.
- CNI misconfiguration can break the entire cluster network.
- Always check routes, MTU, and CNI status when troubleshooting.



SENIOR ENGINEER TIP

"When pod connectivity fails, check it layer by layer: pod IP → node routing → CNI status → encapsulation → MTU → firewall → network policies."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

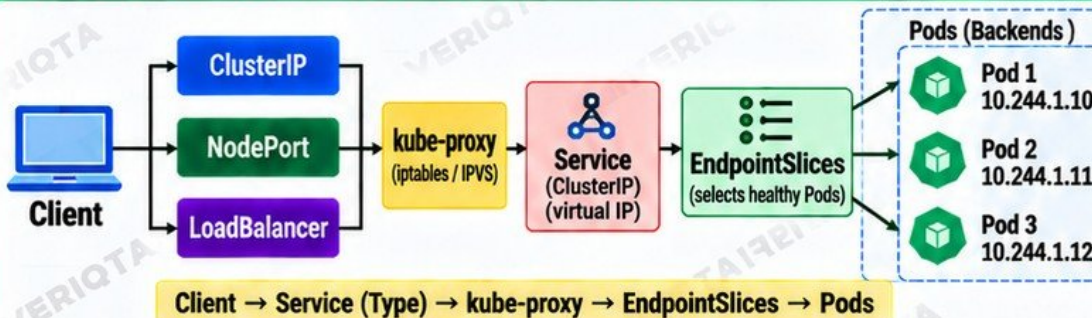
Build
Operate
Improve
Repeat

KUBERNETES SERVICES AND TRAFFIC ROUTING

HOW SERVICES WORK, HOW TRAFFIC FLOWS, AND WHY THINGS SOMETIMES BREAK

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. OVERVIEW: KUBERNETES SERVICE ARCHITECTURE



A Service provides a stable virtual IP and DNS name. kube-proxy routes traffic to healthy Pods using iptables or IPVS.

Key Takeaways

- Services provide stable network access to Pods.
- Different Service types expose services in different ways.
- kube-proxy and EndpointSlices handle traffic routing.
- A Service selects Pods using labels (selectors).
- Even if Pods are healthy, network, configuration or policy issues can still make them unreachable.

2. CLUSTERIP

- Default Service type.
- Exposes the service on a virtual IP inside the cluster.
- Only reachable from within the cluster.
- Used for internal communication between services.

```
# Create a ClusterIP service
kubectl expose deployment app \
  --name app-svc \
  --type ClusterIP
  --port 80
  --target-port 8080
```

3. NODEPORT

- Exposes the service on a static port on every node (30000~32767).
- Accessible via <NodeIP>:<NodePort>.
- Useful for development or when a cloud load balancer is not available.
- Traffic is still routed to a ClusterIP and then to Pods.

```
# Create a NodePort service
kubectl expose deployment app \
  --name app-svc \
  --type NodePort
  --port 80
  --target-port 8080
  --node-port 30080
```

4. LOADBALANCER

- Provision a cloud load balancer (e.g. AWS ELB, Azure LB, GCP LB).
- Exposes the service externally with a public IP.
- Traffic is routed to nodes, then to Pods (like NodePort + ClusterIP).
- Supports ExternalTrafficPolicy for preserving client IP.

```
# Create a LoadBalancer service
kubectl expose deployment app \
  --name app-svc \
  --type LoadBalancer
  --port 80
  --target-port 8080
```

5. ENDPOINTSICES

- Tracks the IP addresses of all Pods that match the Service selector.
- More scalable than the older Endpoints resource.
- Automatically updated as Pods are added or removed.
- kube-proxy uses EndpointSlices to route traffic only to healthy Pods.

```
# View EndpointSlices
kubectl get endpointslices -l kubernetes.io/service-name=app-svc
kubectl describe endpointslices
```

6. KUBE-PROXY (IPTABLES / IPVS)

- kube-proxy programs network rules on each node.
- Can use iptables mode (default) or IPVS mode.
- Routes Service traffic to the correct Pod IPs.
- Performs load balancing (round-robin, etc.).
- Updates rules when EndpointSlices change.

```
# Check kube-proxy mode
kubectl -n kube-system get configmap kube-proxy -o yaml | grep mode

# View iptables rules (iptables mode)
sudo iptables -t nat -L -n | grep <service-ip>
```

7. SERVICE SELECTION

- Services select Pods using labels.
- Only Pods with matching labels receive traffic.
- If no Pods match, the Service has no endpoints and traffic fails.

```
# Example selector in YAML
selector:
  app: web
  tier: frontend
```

8. SESSION AFFINITY

- Supports "ClientIP" session affinity.
- Traffic from the same client IP is routed to the same Pod.
- Useful for stateful applications.

```
sessionAffinity: ClientIP
```

9. EXTERNALTRAFFICPOLICY

- Cluster (default): traffic can go to any node and client IP may be lost (SNAT).
- Local: traffic only goes to Pods on the node and preserves the original client IP.
- Useful for logging, monitoring and source IP based policies.

```
externalTrafficPolicy: Local
```

10. WHY HEALTHY PODS CAN STILL BE UNREACHABLE

- Service selector does not match the Pods.
- No endpoints in EndpointSlices.
- Network policies blocking traffic.
- kube-proxy not updating rules (node issue).
- MTU or CNI networking problems.
- Firewall or cloud security group blocking traffic.
- Using the wrong Service type or port.
- DNS resolution issues.
- Pods are healthy, but not ready (failing readiness probes).

11. USEFUL COMMANDS

COMMAND	PURPOSE
<code>kubectl get svc</code>	List services
<code>kubectl describe svc <name></code>	View service details
<code>kubectl get endpointslices</code>	View EndpointSlices
<code>kubectl get ep</code>	View endpoints (legacy)
<code>kubectl get pods -o wide</code>	View pod IPs and nodes
<code>kubectl logs -n kube-system \</code> <code>-l k8s-app=kube-proxy</code>	Check kube-proxy logs
<code>iptables -t nat -L -n</code>	View iptables rules (node)
<code>ipvsadm -Ln</code>	View IPVS rules (node)



SENIOR ENGINEER TIP

"When a Service is not reachable, always check: selector → endpointslices → kube-proxy → network policies → node firewall → cloud security groups → DNS."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

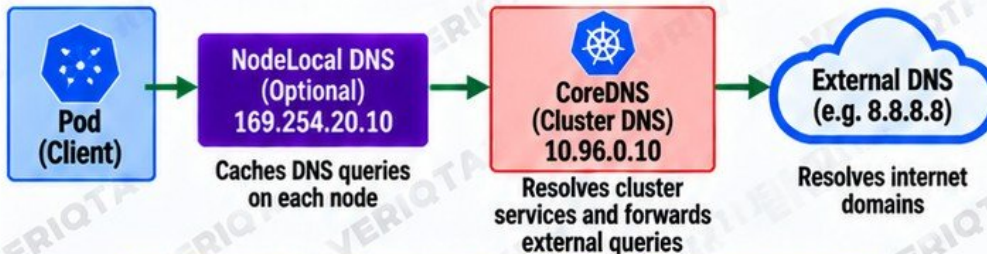
Build
Operate
Improve
Repeat

KUBERNETES DNS AND SERVICE DISCOVERY

HOW COREDNS WORKS, HOW SERVICES ARE RESOLVED, AND HOW TO TROUBLESHOOT DNS ISSUES

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. OVERVIEW: KUBERNETES DNS ARCHITECTURE



Pod → (NodeLocal DNS) → CoreDNS → External DNS

Pods first query NodeLocal DNS (if enabled), then CoreDNS, which resolves cluster services or forwards external queries to upstream DNS servers.

Key Takeaways

- Kubernetes uses CoreDNS for internal DNS resolution.
- Services get DNS names automatically.
- Pods can resolve services using service names (no hardcoded IPs).
- NodeLocal DNS caches queries on each node (improves performance).
- Misconfiguration or high load can cause DNS failures across the cluster.
- DNS is critical for service discovery and application communication.

2. COREDNS

- CoreDNS is the default DNS server in Kubernetes.
- Runs as a Deployment in the kube-system namespace.
- Uses plugins (kubernetes, forward, cache, etc).
- Resolves service names, pod names and external domains.
- Scales horizontally for high availability.

```
# Check CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns

# View CoreDNS configuration
kubectl -n kube-system get configmap coredns -o yaml
```

3. SERVICE DISCOVERY

- Each Service gets a DNS name in the format:
`<service>.<namespace>.svc.cluster.local`
- Pods can resolve services using this name.
- No need for hardcoded IP addresses.
- Works for ClusterIP, headless services and StatefulSets.

```
# Example: resolve a service
nslookup myapp.default.svc.cluster.local

# Test from a pod
kubectl run -it --rm dns-test \
  --image=busybox --restart=Never -- nslookup \
  myapp.default.svc.cluster.local
```

4. SEARCH DOMAINS

- Kubernetes automatically configures search domains for pods.
- Allows short names (e.g. myapp) to be resolved within the namespace.
- Typical search domains:
 - `<namespace>.svc.cluster.local`
 - `svc.cluster.local`
 - `cluster.local`

```
# View pod DNS configuration
kubectl exec <pod> -- cat /etc/resolv.conf

Example output:
search default.svc.cluster.local svc.cluster.local
cluster.local
nameserver 10.96.0.10
options ndots:5
```

5. NDOTS

- The ndots option defines how many dots must be in a name before it's considered an FQDN.
- Default is ndots:5 in Kubernetes.
- If a query has fewer dots, the resolver tries the search domains first.
- Can cause multiple queries and latency for external domains.

```
# Check ndots value
kubectl exec <pod> -- cat /etc/resolv.conf | grep ndots
```

6. DNS QUERY AMPLIFICATION

- With ndots:5, a single query can trigger multiple DNS requests (one per search domain).
- This increases query volume and can overload CoreDNS.
- Common with external lookups (e.g. api.github.com).

```
Example (ndots:5, query for api.github.com):
1. api.github.com.default.svc.cluster.local
2. api.github.com.svc.cluster.local
3. api.github.com.cluster.local
4. api.github.com (final query)
```

7. RESOLVER BEHAVIOR

- Kubernetes uses /etc/resolv.conf inside pods.
- Queries are cached by NodeLocal DNS (if enabled).
- If NodeLocal DNS is not used, every pod query goes directly to CoreDNS.
- Negative responses are also cached (can delay recovery).

```
# Test DNS resolution and timing
kubectl run -it --rm dns-test --image=busybox \
  --restart=Never -- sh
/ # time nslookup kubernetes.default
/ # time nslookup api.github.com
```

8. NODELOCAL DNS

- NodeLocal DNS runs a DNS cache on each node (usually at 169.254.20.10).
- Reduces latency and load on CoreDNS.
- Improves reliability during high DNS traffic.
- Still forwards queries to CoreDNS for cluster domains and to upstream DNS for external domains.

```
# Check NodeLocal DNS
kubectl get daemonset -n kube-system | grep nodelocaldns
kubectl get pods -n kube-system -l k8s-app=nodelocal-dns
```

9. COREDNS SATURATION

- High query volume can cause CoreDNS to become slow or unresponsive.
- Symptoms: timeouts, SERVFAIL, intermittent resolution failures.
- Can be caused by:
 - ndots misconfiguration
 - DNS query amplification
 - Large number of pods
 - External DNS slowness
 - Insufficient CoreDNS replicas or resources.

10. DEBUGGING INTERMITTENT FAILURES

- Check if CoreDNS pods are healthy.
- Look for high CPU/memory usage.
- Check CoreDNS logs for errors.
- Test DNS resolution from inside a pod.
- Verify search domains and ndots.
- Try querying the DNS server directly.
- Use dig or nslookup to compare results.

```
# Useful debugging commands
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=100
kubectl exec -it <pod> -- nslookup <service>
dig @10.96.0.10 <service> +short
```

SENIOR ENGINEER TIP

"When DNS is slow, everything is slow. Always check CoreDNS health, query volume, ndots configuration and upstream DNS before looking at your application."



@veriqta



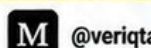
@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

Build
Operate
Improve
Repeat

INGRESS, GATEWAY AND EXTERNAL TRAFFIC

HOW EXTERNAL TRAFFIC ENTERS KUBERNETES, HOW IT IS ROUTED, AND HOW TO TROUBLESHOOT ISSUES

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. OVERVIEW: EXTERNAL TRAFFIC FLOW



Internet → Load Balancer → Gateway/Ingress → Service → Pod

External traffic is received by a load balancer, routed through an Ingress/Gateway, then forwarded to a Kubernetes Service, which sends traffic to healthy Pods.

Key Takeaways

- Ingress and Gateway expose services to external traffic.
- An Ingress Controller or Gateway implements the routing rules.
- External load balancers provide high availability and scalability.
- TLS can be terminated at the LB, Gateway or Ingress Controller.
- Host and path-based routing send traffic to the correct service.
- Source IP preservation may require special configuration.
- Always verify health-check paths.
- Multi-hop failures can happen at any layer, not just in the cluster.

2. INGRESS ARCHITECTURE

- Ingress is a Kubernetes resource that defines HTTP/HTTPS routing rules.
- Requires an Ingress Controller to implement the rules (e.g. NGINX Ingress Controller, AWS ALB Controller).
- Routes traffic based on hostnames, paths and other rules to Kubernetes Services.
- Commonly used for web applications and APIs.

Example: simple Ingress

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port: number: 80
  
```

3. INGRESS CONTROLLER

- Watches Ingress resources and configures the underlying proxy (e.g. NGINX, Envoy, HAProxy).
- Runs as Pods inside the cluster (usually in kube-system namespace).
- Can be exposed via Service (NodePort/LoadBalancer) or integrated with cloud load balancers.
- Handles TLS termination, routing, load balancing and more.

Check Ingress Controller

```

kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
kubectl logs -n ingress-nginx \
  -l app.kubernetes.io/name=ingress-nginx
  
```

4. GATEWAY API CONCEPTS

- Gateway API is the next generation (Kubernetes SIG-Network).
- More flexible and powerful than Ingress.
- Uses Gateway, HTTPRoute, TLSRoute and other resources.
- Supports advanced traffic management (multiple protocols, policies, etc).
- Designed for multiple implementations (e.g. Envoy, NGINX, cloud providers).

Example: Gateway and HTTPRoute

```

apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: web-gateway
spec:
  gatewayClassName: nginx
  listeners:
  - name: http
    protocol: HTTP
    port: 80
  
```

5. EXTERNAL LOAD BALANCERS

- Provisioned by cloud providers (AWS ELB/ALB/NLB, GCP LB, Azure LB).
- Distributes traffic across multiple nodes or Ingress Controller Pods.
- Performs health checks to detect healthy targets.
- Can terminate TLS or pass through to the Ingress Controller.
- Supports different types (L4 or L7).

Check cloud load balancer (AWS example)

```

aws elbv2 describe-load-balancers
aws elbv2 describe-target-groups
aws elbv2 describe-target-health \
  --target-group-arn <arn>
  
```

6. TLS TERMINATION

- TLS can be terminated at:
 - External Load Balancer
 - Ingress Controller
 - Gateway (Gateway API)
- After termination, traffic is usually HTTP inside the cluster (unless using end-to-end TLS).
- Ensure certificates are valid and properly configured.
- Use cert-manager for automated certificate management.

Check TLS certificate

```

openssl s_client -connect \
  app.example.com:443 -servername \
  app.example.com
  
```

7. HOST / PATH ROUTING

- Routes traffic based on:
 - Hostname (e.g. app.example.com)
 - Path (e.g. /api, /web)
- Allows multiple services behind a single external IP.
- Supports rewrites, redirects and advanced rules.

Example path-based routing

```

rules:
- host: app.example.com
  http:
    paths:
    - path: /api
      pathType: Prefix
      backend:
        service:
          name: api-svc
          port: number: 8080
  
```

8. SOURCE IP PRESERVATION

- By default, source IP may be lost due to NAT (especially with cloud load balancers).
- Use **ExternalTrafficPolicy: Local** to preserve the client IP (NodePort/LoadBalancer).
- Some Ingress Controllers add X-Forwarded-For headers.
- Required for accurate logging, rate limiting and security rules.

Example (Service with client IP)

```

apiVersion: v1
kind: Service
metadata:
  name: ingress-svc
spec:
  type: LoadBalancer
  externalTrafficPolicy: Local
  
```

9. HEALTH-CHECK PATHS

- Load balancers and Ingress Controllers use health checks.
- Configure a dedicated health endpoint (e.g. /healthz).
- Ensure the path returns 200 OK and is lightweight.
- Misconfigured health checks can cause traffic to be routed away from healthy pods.

Example (NGINX Ingress annotation)

```

nginx.ingress.kubernetes.io/health-check-path: /healthz
  
```

10. MULTI-HOP FAILURE ANALYSIS

- Failures can occur at any layer:
 1. External load balancer (health check, target down)
 2. Ingress/Gateway (misconfiguration, controller crash)
 3. Service (no endpoints, wrong selector)
 4. Network policies / firewall
 5. Pod (not ready, app error)
- Use logs, metrics and tracing to identify where traffic is failing.
- Always test each hop during troubleshooting.

11. USEFUL COMMANDS

COMMAND	PURPOSE
<code>kubectl get ingress</code>	List Ingress resources
<code>kubectl get gateway</code>	List Gateway API resources
<code>kubectl get svc</code>	List services
<code>kubectl get endpoints</code>	Check service endpoints
<code>kubectl describe ingress</code>	View Ingress details
<code>kubectl logs -n ingress-nginx</code>	Check controller logs
<code>curl -I https://app.example.com</code>	Test from outside cluster
<code>kubectl get pods -o wide</code>	Check pod locations
<code>kubectl describe svc <name></code>	Check Service configuration

SENIOR ENGINEER TIP

"When external traffic issues occur, test each hop: DNS → Load Balancer → Ingress/Gateway → Service → Pod."



@veriqta



@veriqta



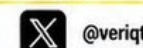
@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

Build
Operate
Improve
Repeat

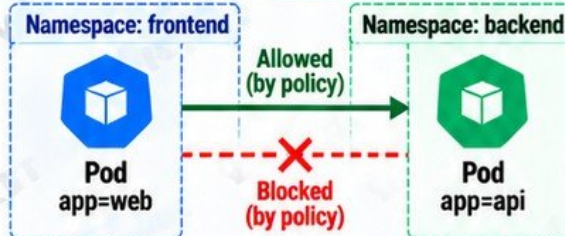
NETWORKPOLICY AND ZERO-TRUST TRAFFIC CONTROL

HOW KUBERNETES NETWORKPOLICY WORKS, HOW TO RESTRICT TRAFFIC, AND HOW TO TROUBLESHOOT ISSUES

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. WHAT IS NETWORKPOLICY?

- NetworkPolicy controls the flow of traffic to and from Pods.
- It operates at Layer 3/4 (IP and port).
- Implemented by the CNI plugin (e.g. Calico, Cilium, Weave, etc).
- Helps enforce zero-trust networking inside the cluster.
- Policies are namespace-scoped.
- If no policy is applied, Pod-to-Pod traffic is allowed by default (most CNIs).



NetworkPolicy allows only the traffic you define.
Everything else is denied.

Key Takeaways

- NetworkPolicy enables zero-trust inside Kubernetes.
- You must explicitly allow required traffic.
- Both ingress and egress can be controlled.
- Policies are enforced by the CNI.
- Don't forget DNS egress.
- Use least-privilege communication.
- Test policies before applying to production.
- Missing rules can break applications even if deployment is successful.

2. DEFAULT ALLOW VS DEFAULT DENY

- By default, most Kubernetes clusters allow all Pod-to-Pod traffic.
- A default deny policy blocks all traffic, then you explicitly allow what is needed.
- Use default deny for production environments (zero-trust).

Example: deny all ingress to a namespace

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: backend
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

3. INGRESS AND EGRESS POLICY

- Ingress policy controls incoming traffic to Pods.
- Egress policy controls outgoing traffic from Pods.
- You can define both in the same policy or separate policies.

Example: allow ingress from frontend

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-api
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              name: frontend
      podSelector:
        matchLabels:
          app: web
      ports:
        - protocol: TCP
          port: 8080
```

4. NAMESPACE ISOLATION

- Use namespaceSelector to control traffic between namespaces.
- Helps isolate environments (dev, staging, prod).
- Combine with podSelector for fine-grained control.

Example: allow specific namespace

```
from:
  - namespaceSelector:
      matchLabels:
        name: monitoring
```

6. CNI ENFORCEMENT

- NetworkPolicy is enforced by the CNI plugin (e.g. Calico, Cilium, Weave).
- Without a compatible CNI, policies will not be enforced.
- Each CNI may have additional features (e.g. FQDN rules, L7 policies).

5. POD SELECTORS

- Select Pods using labels.
- Uses matchLabels or matchExpressions.
- Keep labels simple and consistent.

Example: select Pods with label

```
podSelector:
  matchLabels:
    app: api
    tier: backend
```

7. DNS EGRESS REQUIREMENTS

- Most applications need DNS to resolve service names.
- If you block all egress, you must allow DNS (usually to kube-dns/CoreDNS on port 53).
- Without DNS egress, even internal services will fail.

Example: allow DNS egress

```
egress:
  - to:
      - namespaceSelector:
          matchLabels:
            kube-system: "true"
      podSelector:
        matchLabels:
          k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
```

8. POLICY GAPS

- Forgetting DNS egress.
- Not accounting for external dependencies (e.g. APIs, databases).
- Missing required ports or protocols.
- Incorrect label selection.
- Policies only cover L3/L4 (not HTTP paths).
- Some traffic (e.g. hostNetwork, node-level) may bypass policies.

9. LEAST-PRIVILEGE COMMUNICATION

- Allow only the traffic that is required.
- Use specific namespaces, labels and ports.
- Remove unused rules.
- Regularly review and audit policies.
- Combine with other controls (e.g. mTLS, service mesh, authentication).

10. INCIDENT: DEPLOYMENT SUCCEEDS BUT DEPENDENCIES UNREACHABLE

- The application Pod starts successfully.
- Requests to databases, external APIs or DNS fail.
- Common causes:
 - Missing egress rules
 - Blocked DNS
 - Incorrect namespace or pod selectors
 - CNI policy enforcement
 - Assumed default allow (but default deny is active)
 - External services not allowed
- Troubleshoot using pod logs, nslookup, curl and by reviewing NetworkPolicies.

Example troubleshooting steps

```
kubectl get networkpolicy -A
kubectl describe networkpolicy <name>
kubectl exec -it <pod> -- nslookup <service>
kubectl exec -it <pod> -- curl -v <service>
kubectl get pods -n kube-system -l k8s-app=kube-dns
```



SENIOR ENGINEER TIP

"Start with a default deny policy, then explicitly allow only what is needed. Don't forget DNS, and always test in a non-production environment first."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

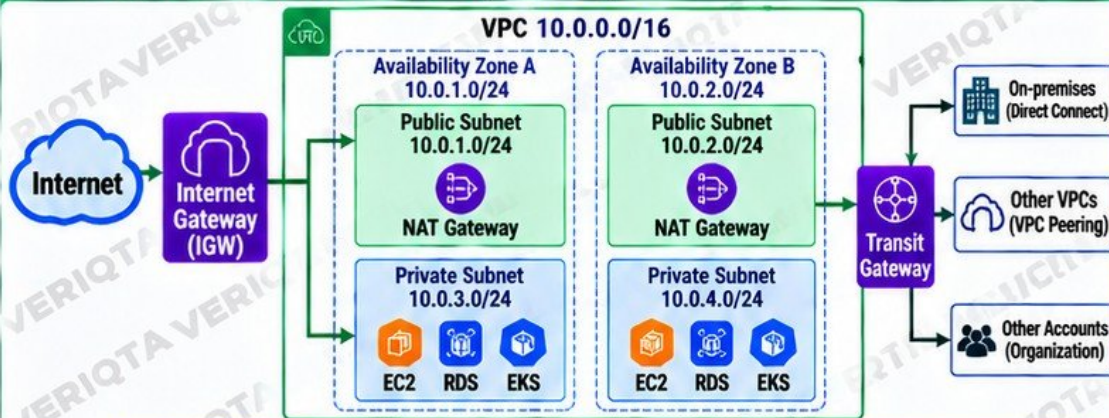
Build
Operate
Improve
Repeat

CLOUD VPC ARCHITECTURE FOR PRODUCTION

HOW TO DESIGN, SECURE AND OPERATE VPC NETWORKING AT SCALE

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. PRODUCTION VPC ARCHITECTURE (EXAMPLE)



A well-designed VPC separates resources, controls traffic, and scales across multiple Availability Zones.

Key Takeaways

- A VPC is a logically isolated network in the cloud.
- Use public and private subnets for tiered security.
- Internet Gateway (IGW) provides internet access to public subnets.
- NAT Gateway allows private subnets to access the internet (outbound only).
- Security groups are stateful (instance level).
- Network ACLs are stateless (subnet level).
- Use peering or transit gateways for multi-VPC connectivity.
- Design for high availability, security and cost optimization.
- Plan CIDRs carefully to avoid overlaps and future rework.

2. CIDR PLANNING

- Choose a CIDR block large enough for current and future growth.
- Use RFC1918 ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16).
- Plan subnet sizes for each tier (web, app, data).
- Avoid overlapping CIDRs (especially when peering or using transit gateway).
- Consider IP reservations for managed services.

Example VPC CIDR:
10.0.0.0/16
Subnets:
10.0.1.0/24 (public-a)
10.0.2.0/24 (public-b)
10.0.3.0/24 (private-a)
10.0.4.0/24 (private-b)

3. PUBLIC VS PRIVATE SUBNETS

- Public subnets: resources with direct internet access (e.g. ALB, bastion hosts).
- Private subnets: no direct internet access (e.g. app servers, databases).
- Use NAT Gateway for outbound internet access from private subnets.
- Spread subnets across multiple Availability Zones for high availability.

Example:
Public Subnet → ALB, NAT Gateway
Private Subnet → EC2, RDS, EKS nodes

4. ROUTE TABLES

- Route tables control where traffic is sent from a subnet.
- Public subnets route 0.0.0.0/0 to the Internet Gateway.
- Private subnets route 0.0.0.0/0 to a NAT Gateway.
- Use separate route tables for public and private subnets.
- Add routes for peering, transit gateways or on-premises networks.

Example (public subnet route):
Destination Target
0.0.0.0/0 igw-xxxxxx
10.0.0.0/16 local

5. INTERNET GATEWAYS

- Provides internet access to resources in public subnets.
- Horizontally scaled and highly available.
- One IGW per VPC.
- Required for inbound and outbound internet traffic.
- Cannot be attached to private subnets directly (use NAT Gateway).

Create an Internet Gateway:
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway \\\n--vpc-id vpc-xxxxxx

6. NAT Gateways

- Allows private subnets to access the internet (outbound only).
- Placed in a public subnet.
- One NAT Gateway per AZ for high availability.
- Managed by cloud provider (no patching).
- Consider cost (charged per hour and per GB).

Create a NAT Gateway:
aws ec2 create-nat-gateway \\\n--subnet-id subnet-xxxxxx

7. SECURITY GROUPS

- Stateful firewall at the instance level.
- Allow specific inbound and outbound traffic.
- Default: deny all inbound, allow all outbound.
- Use least-privilege rules.
- Can reference other security groups (e.g. allow app SG to talk to DB SG).

Example rule (allow HTTPS):
Type: HTTPS (443)
Source: 0.0.0.0/0 (or specific IP)
Target: EC2 instance

8. NETWORK ACLS

- Stateless firewall at the subnet level.
- Control both inbound and outbound traffic.
- Default: allow all traffic.
- Use explicit deny rules for sensitive environments.
- Rules are evaluated in numbered order (lowest number first).

Example (deny SSH):
Rule #: 100
Type: SSH (22)
Action: DENY
Source: 0.0.0.0/0

9. PEERING AND TRANSIT ARCHITECTURES

- VPC Peering: connect two VPCs (simple, not transitive).
- Transit Gateway: hub-and-spoke for multiple VPCs and on-premises.
- Direct Connect: private connection to on-premises.
- Use route tables and security groups to control traffic.
- Avoid overlapping CIDRs.

Example (VPC peering):
aws ec2 create-vpc-peering-connection \\\n--vpc-id vpc-aaa --peer-vpc-id vpc-bbb

10. MULTI-AZ NETWORKING

- Create subnets in at least two AZs.
- Use one NAT Gateway per AZ.
- Deploy load balancers across AZs.
- Ensures high availability and fault tolerance.
- Design for AZ failure (test it).

Best practice:
Never put all critical resources in a single AZ.

11. OVERLAPPING CIDRS

- Causes routing conflicts and connectivity issues.
- Common in mergers, multi-account setups or hybrid cloud.
- Use CIDR planning and IPAM tools.
- Consider NAT, routing or network address translation to solve overlaps.
- Plan early to avoid costly rework.

If overlap exists:
Use NAT, Transit Gateway with CIDR mapping, or renumber the network.

12. COST AND BLAST-RADIUS TRADE-OFFS

- More AZs and NAT Gateways increase cost but improve resilience.
- Centralized vs distributed architectures affect cost and failure impact.
- Use least-privilege network access to reduce blast radius.
- Segment environments (dev, staging, prod) into separate VPCs or accounts.
- Balance security, availability and cost based on business requirements.

Key question:
What is the impact if this component fails?

COMMON MISTAKES

- Using overlapping CIDRs.
- Missing route table entries.
- Misconfigured security groups (too open).
- Forgetting NAT Gateway for private subnets.
- Single-AZ deployments in production.
- Not testing network failures.
- Allowing unnecessary internet access.
- Ignoring cost implications.
- Not documenting the architecture.

SENIOR ENGINEER TIP

"Design your VPC like a security boundary, not just a network. Use least-privilege access, multiple AZs and proper segmentation to reduce risk and improve resilience."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

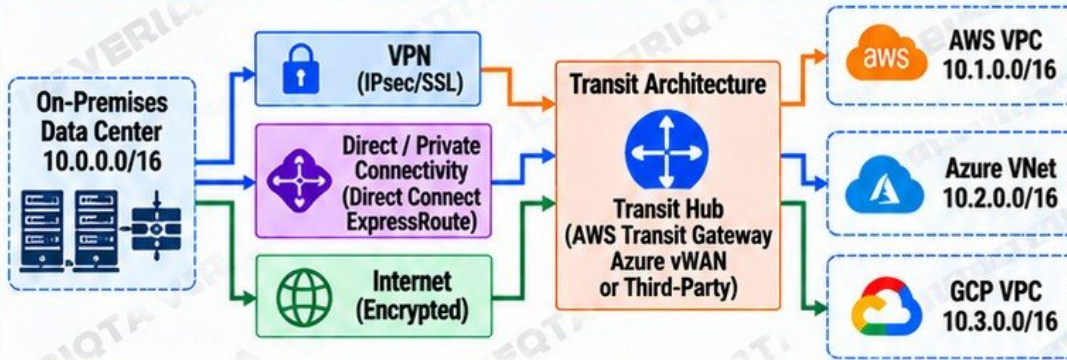
Build
Operate
Improve
Repeat

HYBRID, PRIVATE AND MULTI-CLOUD CONNECTIVITY

HOW TO CONNECT ENVIRONMENTS SECURELY, ROUTE TRAFFIC, AND DESIGN FOR RESILIENCE

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. OVERVIEW: HYBRID AND MULTI-CLOUD CONNECTIVITY ARCHITECTURE



Hybrid and multi-cloud connectivity uses VPN or private links to connect environments, with a transit architecture to route traffic securely, at scale and with high availability.

Key Takeaways

- Use VPN or private connectivity to connect on-premises and cloud environments.
- A transit architecture simplifies routing, security and scalability.
- BGP enables dynamic route exchange between environments.
- Plan for overlapping networks and DNS integration early.
- Design for high availability with failover and multiple paths.
- Consider latency, bandwidth, cost and operational complexity.
- Choose between centralized transit or distributed connectivity based on your organization's needs.

2. VPN

- VPN (IPsec or SSL) creates an encrypted tunnel over the internet.
- Common for site-to-cloud or site-to-site connectivity.
- Suitable for smaller environments or as a backup link.
- Higher latency compared to private connectivity.

Example: AWS Site-to-Site VPN

```
aws ec2 create-vpn-connection \
--type ipsec.1 \
--customer-gateway-id cgw-xxxxxx \
--vpn-gateway-id vgw-xxxxxx
```

3. DIRECT / PRIVATE CONNECTIVITY

- Uses dedicated private links (AWS Direct Connect, Azure ExpressRoute, Google Cloud Interconnect).
- Provides lower latency, higher bandwidth, and more consistent performance than VPN.
- Often used for production and mission-critical workloads.

Example: AWS Direct Connect

```
aws directconnect create-connection \
--bandwidth 1Gbps \
--location "Equinix" \
--connection-name prod-dx
```

4. TRANSIT ARCHITECTURE

- A central hub (e.g. AWS Transit Gateway, Azure vWAN, third-party) connects multiple networks.
- Simplifies routing, security policies, and management.
- Supports hybrid and multi-cloud connectivity.
- Can integrate with on-premises networks via VPN or Direct Connect.

Example: AWS Transit Gateway

```
aws ec2 create-transit-gateway \
--description "prod-tgw" \
--options AmazonSideAsn=64512
```

5. BGP CONCEPTS

- BGP (Border Gateway Protocol) exchanges routes between networks.
- Enables dynamic routing and failover.
- Used in VPN, Direct Connect and multi-cloud setups.
- Supports route filtering, route propagation and policy control.

Example: BGP configuration (on a router)

```
router bgp 65001
neighbor 169.254.1.1
remote-as 65000
network 10.0.0.0/16
```

6. ROUTE PROPAGATION

- Routes learned via BGP or static routes are propagated to connected networks.
- Control which routes are shared (i.e. use route tables, route filters, or BGP policies).
- Avoid advertising unnecessary routes.

Example: AWS route table

```
aws ec2 create-route \
--route-table-id rtb-xxxxxx \
--destination-cidr-block 10.0.0.0/16 \
--transit-gateway-id tgw-xxxxxx
```

7. OVERLAPPING NETWORKS

- Overlapping CIDRs (e.g. 10.0.0.0/16 in both on-prem and cloud) cause routing conflicts.
- Use NAT, CIDR translation or separate address ranges.
- Plan IP addressing early to avoid future issues.

Example: NAT for overlapping networks

```
# Use NAT or network address translation at the edge
# or re-address one environment to a unique CIDR range
```

8. DNS INTEGRATION

- Integrate DNS across on-prem and cloud environments.
- Use conditional forwarding or private DNS zones.
- Ensure services can resolve names across all environments.
- Consider split-horizon DNS for internal and external resolution.

Example: Route 53 Resolver conditional forwarder

```
aws route53resolver create-resolver-rule \
--domain-name corp.local \
--target-ips Ip=10.0.0.10 \
--rule-type FORWARD
```

9. FAILOVER

- Design multiple connections (VPN + Direct Connect).
- Use dynamic routing (BGP) for automatic failover.
- Test failover regularly.
- Monitor route changes and link health.

Example: BGP failover

```
# If primary link fails, BGP
# withdraws the route and traffic
# automatically shifts to the
# secondary link based on
# routing policy.
```

10. LATENCY AND BANDWIDTH

- Latency affects application performance (especially for database and real-time apps).
- Private links provide lower and more consistent latency.
- Choose appropriate bandwidth based on workload and growth (consider peak traffic).
- Monitor and optimize using tools like CloudWatch, Azure Monitor, or third-party solutions.

11. ARCHITECTURE TRADE-OFF

Centralized Transit (Hub-and-Spoke)

- Easier management and security control.
- Centralized routing, policies and monitoring.
- Scales well for many environments.
- Potential single point of failure (mitigate with high availability).
- Best for most organizations.

Distributed Connectivity (Mesh/Point-to-Point)

- Direct connections between environments.
- Lower latency for certain use cases.
- More complex to manage at scale.
- Inconsistent security policies.
- Can be more expensive.
- Useful for specific high-performance or regulatory requirements.

Common Mistakes

- Using overlapping CIDRs.
- Missing DNS integration.
- Not planning for failover.
- Advertising too many routes.
- Weak security controls.
- Ignoring latency and bandwidth requirements.
- Overcomplicating the architecture.
- Not testing connectivity regularly.

SENIOR ENGINEER TIP

"Design your connectivity for failure, not for perfection. Use simple, well-documented architectures, monitor everything, and test failover regularly."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

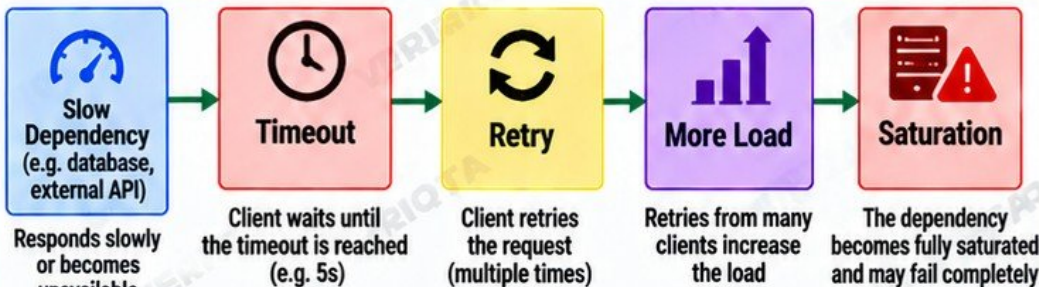
Build
Operate
Improve
Repeat

TIMEOUTS, RETRIES AND CASCADING NETWORK FAILURES

HOW SMALL DELAYS BECOME BIG INCIDENTS, AND HOW TO DESIGN RESILIENT SYSTEMS

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. CASCADING FAILURE FLOW (EXAMPLE)



Retries can turn a small slowdown into a full outage if not properly controlled.

Key Takeaways

- Timeouts, retries and poor configuration can cause cascading failures.
- Set appropriate timeouts for connection, read and idle time.
- Use retry budgets, exponential backoff and jitter.
- Avoid retry storms and unbounded retries.
- Use connection pools and circuit breakers to protect your services.
- Monitor and optimize tail latency.
- Design for failure; every dependency will be slow or unavailable at some point.
- Test your system under real failure conditions.

2. CONNECTION TIMEOUT

- Time to establish a TCP connection (3-way handshake).
- If the service is unreachable, the connection fails after this time.
- Set a reasonable value (e.g. 1-5s).

Example (curl):

```
curl --connect-timeout 3 \
https://api.example.com
```

3. READ TIMEOUT

- Time to wait for data after a connection is established.
- Protects against a slow or unresponsive server.
- Set based on expected response time (e.g. 5-30s).

Example (Python requests):

```
requests.get("https://api.example.com",
timeout=(3, 10)) # (connect, read)
```

4. IDLE TIMEOUT

- Time to keep an idle connection open with no activity.
- If no data is sent or received, the connection is closed.
- Important for load balancers, proxies and connection pools.

Example (Nginx):

```
keepalive_timeout 60;
```

5. RETRY BUDGETS

- Limit the total number of retries per request or time period.
- Prevents infinite retries and protects dependencies.
- Allows some retries for resilience without causing overload.

Example logic:

```
max_retries = 3
retry_budget = 20% of original
traffic
```

6. EXPONENTIAL BACKOFF

- Increase the wait time between retries (e.g. 1s, 2s, 4s, 8s).
- Gives the dependency time to recover.
- Reduces load during incidents.

Example (Python):

```
import time
for i in range(3):
    try:
        call_service()
        break
    except Exception:
        time.sleep(2 ** i) # 1, 2, 4
```

7. JITTER

- Adds randomness to retry delays (e.g. 1-3s instead of exactly 2s).
- Prevents all clients from retrying at the same time.
- Reduces the risk of retry storms.

Example (Python):

```
import random, time
delay = random.uniform(1, 3)
time.sleep(delay)
```

8. RETRY STORMS

- Happen when many clients retry at the same time.
- Can quickly overwhelm a slow or failing dependency.
- Use retry budgets, backoff, jitter and circuit breakers to prevent this.



9. CONNECTION POOLS

- Reuse existing connections instead of creating a new connection for every request.
- Improves performance and reduces resource usage.
- Set limits to avoid exhausting connections.

Example (Python requests):

```
import requests
from requests.adapters
import HTTPAdapter
session = requests.Session()
adapter = HTTPAdapter(
pool_connections=10,
pool_maxsize=50)
session.mount("http://", adapter)
```

10. CIRCUIT BREAKERS

- Stop sending requests to a failing service after a certain number of failures.
- Allow the service time to recover.
- Prevent cascading failures.
- Common states: Closed → Open → Half-Open.



Example: Use libraries like Resilience4j, Hystrix (legacy) or built-in cloud tools.

11. TAIL LATENCY

- The 95th or 99th percentile of response times is often much higher than the average.
- High tail latency can cause timeouts, retries and a poor user experience.
- Monitor and optimize tail latency, not just the average.

Example latency distribution:

```
P50: 100 ms
P90: 300 ms
P95: 800 ms
P99: 2,000 ms ← This causes timeouts!
```

Design for the worst case, not just the average.

COMMON MISTAKES

- Setting timeouts too high or too low.
- Unbounded or aggressive retries.
- No jitter in retry logic.
- Ignoring connection pool limits.
- No circuit breakers.
- Not monitoring tail latency.
- Retrying non-idempotent operations (e.g. creating a payment twice).
- Not testing failure scenarios.
- Assuming dependencies are always fast and available.

SENIOR ENGINEER TIP

"Timeouts and retries are not just client settings, they are part of your system's reliability design. Use them wisely, test them under load, and always assume something will fail."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

Build
Operate
Improve
Repeat

NETWORK OBSERVABILITY AND PACKET-LEVEL DEBUGGING

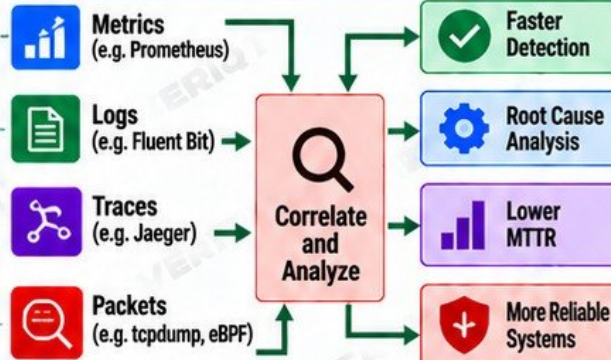
HOW TO SEE, MEASURE, AND TROUBLESHOOT NETWORK ISSUES LIKE A PRO

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. NETWORK OBSERVABILITY OVERVIEW

- Network observability gives you visibility into how traffic flows, where it fails, and why.
- Use metrics, logs, traces and packet-level data to get the full picture.
- Helps you detect issues early, reduce MTTR and prevent recurring incidents.
- No single tool shows everything – correlate multiple data sources.

"If you can't see it, you can't fix it."



Key Takeaways

- Use RED, connection, DNS and LB metrics to detect issues.
- Packet-level tools show the truth when metrics are not enough.
- Always correlate metrics, logs, traces and packets.
- Understand normal behavior so you can spot anomalies.
- Use eBPF for deep visibility in modern environments.
- Flow logs help with cross-VPC and cloud network debugging.
- Observability is essential for performance, security and reliability.
- Practice with real incidents and capture evidence.

2. RED SIGNALS

- RED = Rate, Errors, Duration.
- Measure at each service, load balancer and dependency.
- Helps identify user impact and performance degradation.

Example (Prometheus query):

```
# Request rate
rate(http_requests_total[5m])

# Error rate
rate(http_requests_total(status=~"5..")[5m])

# Duration (latency)
histogram_quantile(0.95,
rate(http_request_duration_seconds_bucket[5m]))
```

3. CONNECTION METRICS

- Track new connections, active connections and connection rates.
- Helps identify spikes, exhaustion and DoS attacks.
- Monitor connection limits on services, load balancers and firewalls.

Example (Linux):

```
ss -s # summary
ss -ant | wc -l # total connections
ss -ant state established | wc -l # active connections
```

4. DNS LATENCY

- High DNS latency can cause slow application responses.
- Check query time, failure rate and resolver health.
- Monitor CoreDNS or external DNS services.

Example (dig):

```
dig api.example.com
# With timing
dig api.example.com +stats
# Continuous test
while true; do dig api.example.com
+stats | grep "Query time"; sleep 1; done
```

5. RETRANSMISSIONS & PACKET LOSS

- Retransmissions indicate network issues, congestion or dropped packets.
- Packet loss reduces performance and can break connections.
- Monitor at host, network device and cloud provider level.

Example (Linux):

```
# TCP retransmissions
ss -it | grep retrans
# Packet loss test
mtr -rwz 5 api.example.com
# Interface statistics
ip -s link show eth0
```

6. SOCKET STATES

- Socket states show the lifecycle of TCP connections.
- Look for high SYN_SENT, TIME_WAIT or CLOSE_WAIT.
- Helps detect connection issues and resource leaks.

Example (Linux):

```
ss -ant
# Count by state
ss -ant | awk '{print $1}' |
sort | uniq -c
```

7. CONNTRACK

- Conntrack tracks active connections in the kernel.
- Useful for NAT, service meshes and firewalls.
- Can show connection limits and dropped entries.

Example (Linux):

```
# View conntrack entries
conntrack -L | head
# Check count
conntrack -S
# Filter by IP
conntrack -L | grep 10.0.0.10
```

8. FLOW LOGS

- Flow logs give visibility into network traffic (VPC, subnet, security group, etc.).
- Useful for security, auditing and troubleshooting.
- Available in AWS, Azure, GCP and also on-prem.

Example (AWS VPC Flow Logs):

```
Fields: srcaddr, dstaddr, srcport,
dstport, protocol, action
Analyze with:
Athena / CloudWatch / SIEM
```

9. LOAD BALANCER METRICS

- Monitor request count, latency, error rate, healthy/unhealthy targets and connection metrics.
- Helps detect backend issues and traffic spikes.
- Check metrics for ALB, NLB, ELB, Azure LB, GCP LB, etc.

Example (AWS ALB metrics):

```
- RequestCount
- TargetResponseTime
- HTTPCode_Target_5XX_Count
- HealthyHostCount
- UnHealthyHostCount
```

10. eBPF VISIBILITY

- eBPF provides deep, low-overhead visibility into network activity.
- Can trace connections, DNS queries, latency and application behavior.
- Tools: Cilium, Pixie, Parca, bpftrace.

Example (bpftrace):

```
bpftrace -e 'tracepoint:tcp:tcp_retransmit_skb
{ printf("Retransmit: %s\n", comm); }'
```

11. COMMON DEBUGGING COMMANDS

- tcpdump - Capture and analyze packets
- ss - Check socket states
- dig - Test DNS resolution
- curl - Test HTTP/HTTPS endpoints
- traceroute - Show network path (hop-by-hop)
- mtr - Continuous traceroute with stats

Example:

```
tcpdump -i eth0 -nn host 10.0.0.10
curl -v https://api.example.com
mtr -rwz 5 api.example.com
```

12. CORRELATE EVIDENCE

- Use metrics to detect the issue.
- Check logs for error messages.
- Use traces to find where latency occurs.
- Capture packets to confirm what is happening.
- Correlate timestamps across all sources.
- Build a timeline of the incident.



SENIOR ENGINEER TIP

"Don't guess. Collect evidence. Correlate data. Find the real cause."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

Build
Operate
Improve
Repeat

SENIOR PRODUCTION NETWORK INCIDENT

INTERMITTENT API FAILURES DURING PEAK TRAFFIC - A COMPLETE TROUBLESHOOTING GUIDE

ARCHITECT
TROUBLESHOOT
OPTIMIZE
SECURE
SCALE

1. SCENARIO: INTERMITTENT API FAILURES

- Users report intermittent API failures during peak traffic (e.g. 5XX, timeouts).
- Some requests succeed, some fail.
- Impacts customers, revenue and trust.
- Goal: find root cause, mitigate and prevent recurrence.

This requires end-to-end network troubleshooting, not guesswork.

2. DEFINE BLAST RADIUS

- Identify what is affected.
- Which services, regions, users?
- Is it partial or complete?
- Check error rates, geography, customer segments.
- Understand business impact.

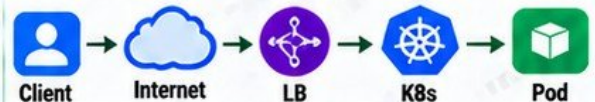
Example:

Only /api/orders failing from US-East, not EU.
Impacts 20% of users.

3. MAP THE COMPLETE REQUEST PATH

- Draw the full path from client to backend.
- Include: DNS → LB → CDN (if any) → Firewall → Ingress/Proxy → Service → Pod → Dependencies.
- Identify all network hops and components.

Know the path before you troubleshoot.



4. CHECK RECENT CHANGES

- Look for recent deployments, config changes, DNS changes, LB updates, scaling events.
- Check change management records and Git history.
- Correlate with incident start time.

Tools: kubectl rollout history
git log, CI/CD, cloud console

5. INSPECT DNS

- Check resolution time, failures and returned IPs.
- Compare from inside and outside the cluster/VPC.
- Look for high latency or NXDOMAIN.

Commands:

```
dig api.example.com
+time +stats
nslookup api.example.com
```

6. INSPECT TCP/SOCKET STATE

- Check connection establishment, retransmissions and socket states.
- Look for SYN_SENT, TIME_WAIT, CLOSE_WAIT.
- Identify connection exhaustion.

Commands:

```
ss -s
ss -ant | grep :443
netstat -s | grep retrans
```

7. CHECK NAT AND CONNTRACK

- Verify NAT gateway health.
- Check conntrack table for dropped or exhausted entries.
- Look for SNAT port exhaustion (especially during high traffic).

Commands:

```
sudo conntrack -S
sudo conntrack -L | wc -l
Check cloud NAT metrics
```

8. CHECK LOAD BALANCER HEALTH

- Verify target health and unhealthy targets.
- Check latency, 5XX rates, TCP resets.
- Ensure health check paths are correct.

Tools:

AWS ALB/NLB metrics
Azure LB logs
GCP LB diagnostics

9. CHECK KUBERNETES NETWORKING

- Verify Service, Endpoints, EndpointSlices.
- Check CNI, NetworkPolicy, kube-proxy.
- Ensure Pods are healthy and reachable.

Commands:

```
kubectl get endpoints
kubectl get svc
kubectl get pods -o wide
kubectl describe networkpolicy
```

10. INSPECT LATENCY, RETRIES AND TIMEOUTS

- Check latency at each hop.
- Look for increased retries, timeouts or retry storms.
- Analyze client and server metrics (RED).

Tools:

Prometheus / Grafana
APM (Datadog, New Relic)
Application logs

11. CAPTURE PACKETS

- Capture packets at key points (client, node, pod).
- Look for DNS issues, TCP resets, TLS handshake failures.
- Compare working vs failing requests.

Commands:

```
tcpdump -i any -nn host <ip>
wireshark / pkt analysis
kubectl sniff (or tcpdump in pod)
```

12. FORM AND TEST HYPOTHESES

- Base on evidence, form likely causes (DNS, routing, NAT, LB, etc).
- Test each hypothesis with real data.
- Don't jump to conclusions.

Example hypothesis:

"SNAT port exhaustion on NAT gateway during peak traffic."

13. IDENTIFY ROOT CAUSE

- Use evidence from metrics, logs, traces and packets.
- Prove the exact reason for failure.
- Document the root cause clearly.

Example:

NAT gateway SNAT port exhaustion caused intermittent connection failures to the external API.

14. MITIGATE SAFELY

- Apply a safe, minimal fix.
- Avoid making the situation worse.
- Validate the fix in real time.

Examples:

Increase NAT gateway capacity
Fix health check path
Adjust timeouts or connection pool

15. VERIFY RECOVERY

- Confirm error rates return to normal.
- Check all regions and services.
- Monitor for stability.
- Close the incident only when fully resolved.

Use dashboards and real user monitoring to confirm.

16. DESIGN PREVENTION CONTROLS

- Add monitoring and alerting.
- Set better timeouts, retries and connection limits.
- Improve capacity planning.
- Automate detection of similar issues.
- Update runbooks and share learnings.

INCIDENT INVESTIGATION PROGRESSION (INTENTIONAL ORDER)



SENIOR ENGINEER TIP

"Follow the evidence, not assumptions. Slow down, verify each layer, and document everything. A systematic approach always beats guesswork."



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KNOWLEDGE BUILDS BETTER ENGINEERS

Build
Operate
Improve
Repeat